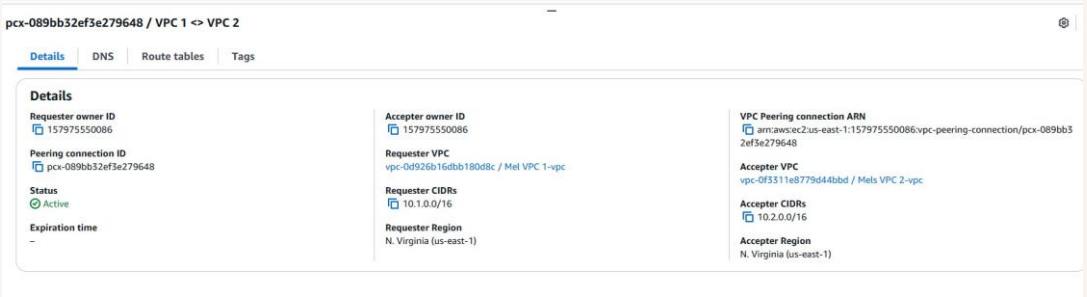
VPC Peering

Melvin green



Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is AWS's foundational networking service that lets us create our own networks, control traffic flow and security, and organize our resources into public and private subnets

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to set up a multi-VPC architecture by creating two VPCs, establishing a peering connection, and updating security group rules to run a successful connectivity test to validate my VPC peering setup.

One thing I didn't expect in this project was...

Here's the corrected version: "One thing I did not expect in this project was that a public IPv4 address would be required for EC2 Instance Connect to work. I also learned that an Elastic IP can assign a static IPv4 address to a resource.

This project took me...

This project took me 1.5 hours.

In the first part of my project...

Step 1 - Set up my VPC

In this step, I will be using the VPC resource map to launch two VPCs within a couple of minutes.

Step 2 - Create a Peering Connection

In this step, I will set up the VPC peering connection, which is a VPC component designed to correctly connect two VPCs together.

Step 3 - Update Route Tables

In this step, I will set up routes from VPC 1 to VPC 2 and vice versa.

Step 4 - Launch EC2 Instances

In this step, I will launch an EC2 instance in each of the VPCs so I can directly connect to my instances later and test the VPC peering connection.

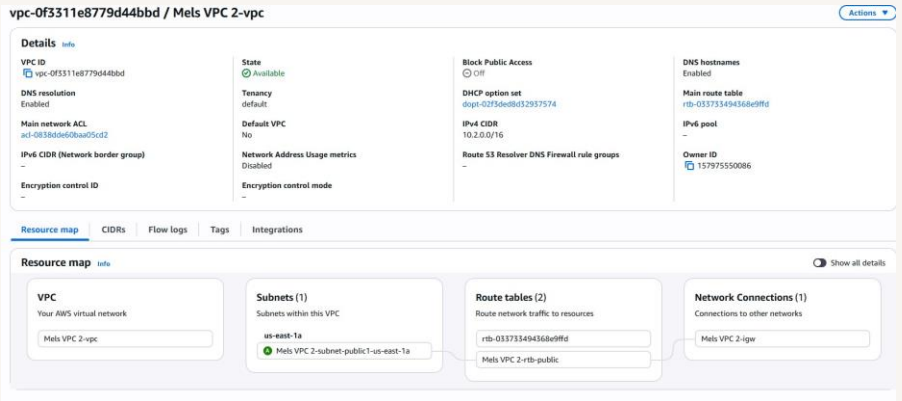
Multi-VPC Architecture

I started my project by launching two different VPCs, each with its own public subnet. I created them by using the 'VPC and more' tab within the Create VPC tab.

The CIDR blocks for VPC 1 and VPC 2 are 10.1.0.0/16 and 10.2.0.0/16. They must be unique because they are two different VPCs on two different networks. The goal of this project is to have them communicate with each other using VPC peering. Once this connection is set up, the route tables need unique addresses for correct routing across the VPCs.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances because we plan to connect to them later in the project using a solution that handles key pair creation and management for us.

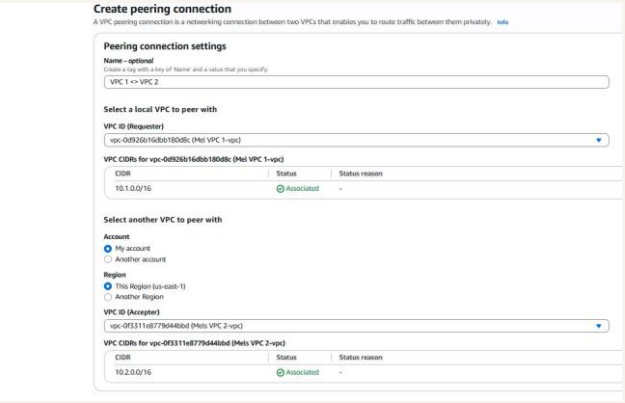


VPC Peering

A VPC peering connection is a direct connection between two VPCs. A peering connection lets VPCs and their resources route traffic between them using their private IP addresses, meaning data can be transferred between VPCs without going through the public internet.

Without a peering connection, data transfers between VPCs would use the resources' public addresses, meaning VPCs would have to communicate over the public internet.

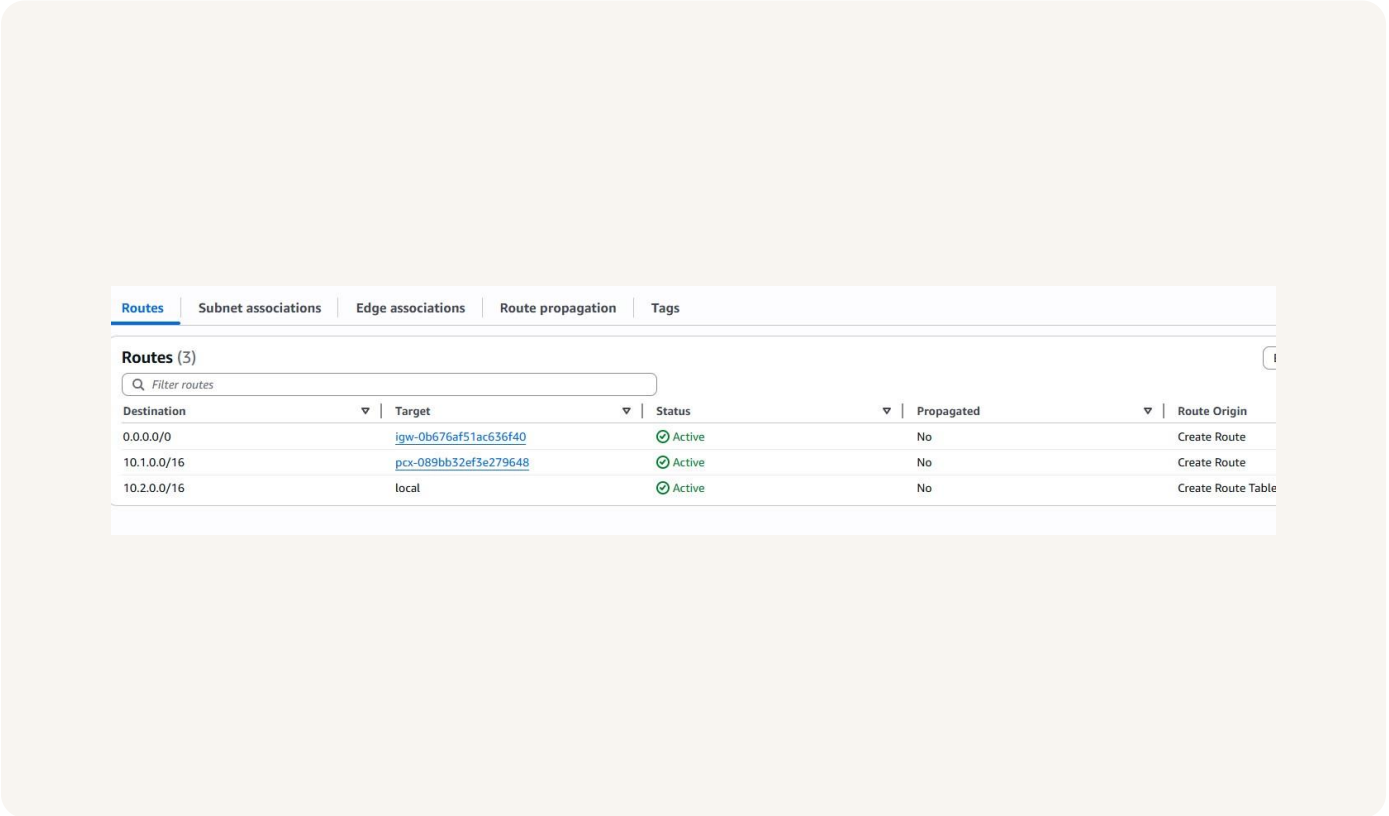
In VPC peering, the Accepter is the VPC that receives a peering connection request. The Accepter can either accept or decline the invitation, meaning the peering connection is not actually established until the other VPC agrees to it. In VPC peering, the Requester is the VPC that initiates the peering connection. As the Requester, it sends the other VPC an invitation to connect.



Updating route tables

After accepting a peering connection, the VPCs' route tables need to be updated because, without routes in the route tables, the VPCs' resources will not know where to direct their traffic, even though they are connected through VPC peering.

My VPCs' new routes have a destination of the other VPC's CIDR block: 10.2.0.0/16 for VPC 1 and 10.1.0.0/16 for VPC 2. The routes' target is the VPC peering connection.



In the second part of my project...

Step 5 - Use EC2 Instance Connect

In this step, I will use an EC2 instance to connect directly to the first EC2 instance. I need to do this because I need to use my EC2 instance for my connectivity test later in this project.

Step 6 - Connect to EC2 Instance 1

In this step, I will reattempt my connection with my instance -Mel's VPC 1 and resolve another error preventing me from using EC2 Instance Connect to directly connect to the instance.

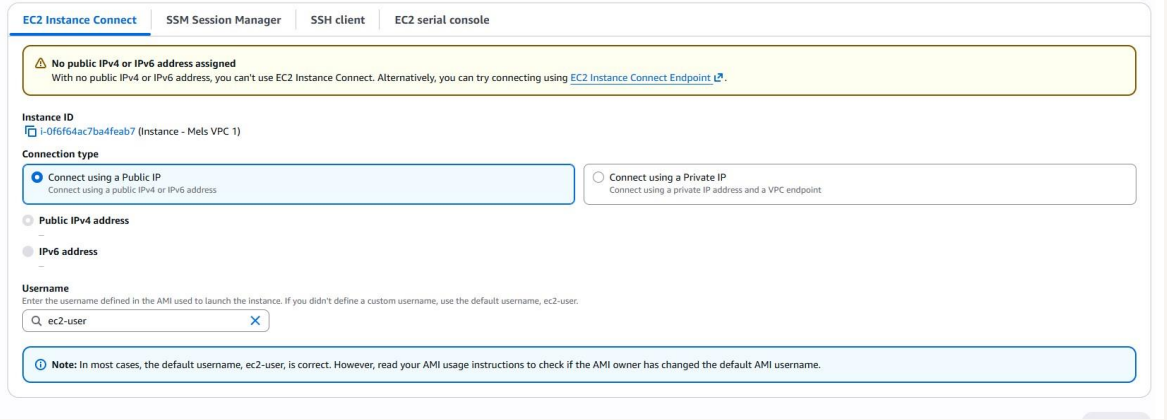
Step 7 - Test VPC Peering

In this step, I will use Instance - Mel's VPC 1 to attempt a direct connection with Instance - Mel's VPC 2 to validate my network setup.

Troubleshooting Instance Connect

Next, I used EC2 Instance Connect to Instance - Mels VPC 1 just by using the AWS Management Console

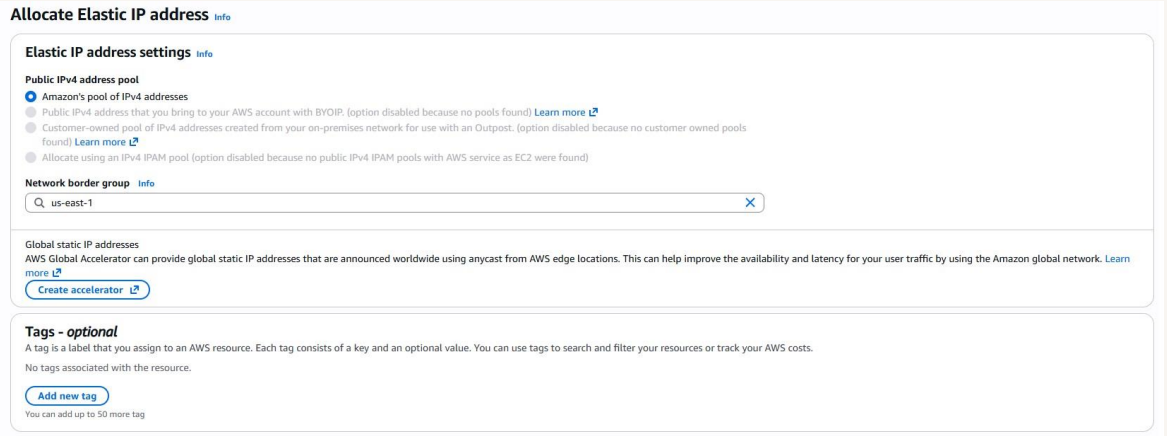
I wasn’t able to use EC2 Instance Connect because the instance did not have a public IPv4 address. For EC2 Instance Connect to work, the instance must have a public IPv4 address and be in a public network."



Elastic IP addresses

To resolve this error, I set up Elastic IP addresses. Elastic IP addresses are public IPV4 static addresses that we can request from our AWS account and delegate to specific resources.

Associating an Elastic IP address resolved the error because it gives the EC2 instance a public IP address, fulfilling the requirements for EC2 Instance Connect to work.



Troubleshooting ping issues

To test VPC peering, I ran the ping command to 10.2.xxx.xxx, which is the private IPv4 address of the other VPC's instance, Mel's VPC 2.

A successful ping test would validate my VPC peering connection because this ping test would not get any replies from other EC2 instances if the peering connection did not successfully connect our two VPCs.

I had to update my second EC2 instance's security group because it was not allowing ICMP traffic, which is the traffic type used by the ping command. I added a new rule that allows ICMP traffic coming in from any resource within VPC 1.